

# Lab Experiment: Bank Account Management System

Implementation of Object-Oriented Programming Concepts in Java

**Course:** Object Oriented Programming Lab

**Language:** Java (JDK 8+)

**Document Code:** LAB-PRG-01

## 1. Objective

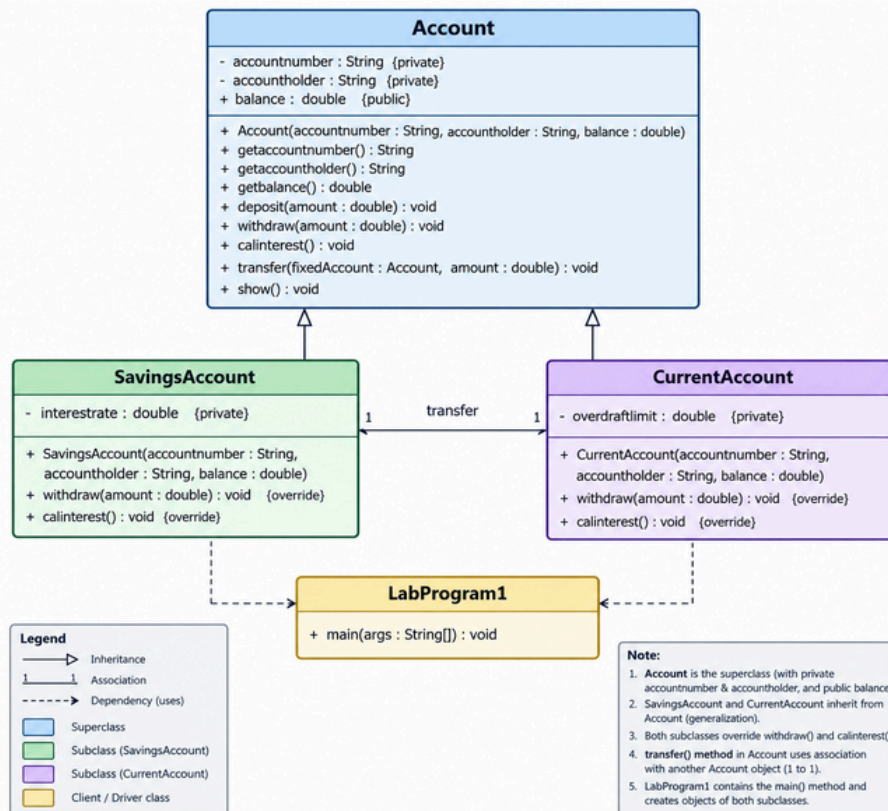
design and implement a **Bank Account Management System** in Java demonstrating key Object-Oriented Programming (OOP) concepts including:

- **Encapsulation:** Hiding internal data members using private access specifiers and exposing access via getter/setter methods.
- **Inheritance:** Deriving specialized classes (SavingsAccount, CurrentAccount) from a base class (Account).
- **Polymorphism / Method Overriding:** Dynamic method dispatching for operations like withdraw() and calinterest() specific to account types.
- **Inter-Account Transfers:** Reusing foundational account behaviors to perform transactions between account objects.

## 2. System Requirements

| Component | Type             | Minimum Specification                 | Recommended Specification                |
|-----------|------------------|---------------------------------------|------------------------------------------|
| Hardware  | Processor        | Intel Core i3 / AMD Ryzen 3 (1.5 GHz) | Intel Core i5 / AMD Ryzen 5 (2.0 GHz+)   |
|           | RAM              | 2 GB                                  | 4 GB or higher                           |
|           | Storage          | 500 MB free hard disk space           | 1 GB free SSD space                      |
| Software  | Operating System | Windows 7 / Linux / macOS             | Windows 10/11 or Ubuntu 20.04 LTS        |
|           | JDK Environment  | Java Development Kit (JDK) 8          | Java SE Development Kit 17 LTS or 21 LTS |
|           | IDE / Editor     | Command Line (javac / java)           | Eclipse, IntelliJ IDEA, or VS Code       |

### 3. UML Class Diagram



### 4. Algorithm

The program follows an execution flow summarized in the steps below:

#### 1. Class Definition & Initialization:

- Define the base class Account containing attributes accountnumber, accountholder, and balance.
- Implement constructor and getter methods. Implement basic deposit() and a high-level transfer() mechanism.

#### 2. Derived Class Implementations:

- Create SavingsAccount extending Account. Override withdraw() to check if balance is sufficient, and override calinterest() to add interest to the balance.
- Create CurrentAccount extending Account. Override withdraw() to allow withdrawal up to (balance + overdraftlimit), and override calinterest() to apply maintenance charges.

#### 3. Main Driver Execution (LabProgram1):

- Instantiate CurrentAccount (ca) with account number "UB 24311", holder "kumar", and balance 4500.
- Instantiate SavingsAccount (sa) with account number "sb 24533", holder "heamnth", and balance 5000.
- Display initial account states using show().
- Perform transactions:
  - Deposit 2000 into <sup>ca</sup>.
  - Withdraw 2500 from <sup>ca</sup>.

- Calculate charges/interest for  $a$  (adds maintenance charge 50.0).
  - Withdraw 3000 from  $a$ .
  - Calculate interest for  $a$  (adds balance \* interestrate).
  - Transfer 1000 from  $a$  to  $c$ .
- Display the final updated state for both accounts.

## 5. Compilation & Execution Instructions

Follow these steps to compile and execute the program from the terminal or command prompt:

**Note:** Save the source code in a file named exactly `LabProgram1.java` since the public class name is `LabProgram1`.

### Step 1: Open Terminal / Command Prompt

Navigate to the directory containing `LabProgram1.java`:

```
cd path/to/your/java/files
```

### Step 2: Compile the Java Source File

Run the Java Compiler (`javac`):

```
javac LabProgram1.java
```

*If compilation succeeds, bytecode class files (`Account.class`, `SavingsAccount.class`, `CurrentAccount.class`, and `LabProgram1.class`) will be generated.*

### Step 3: Execute the Program

Run the compiled Java class using the Java Virtual Machine (`java`):

```
java LabProgram1
```

## 6. Source Code

```
class Account {
    private String accountnumber; // details are hidden
    private String accountholder;
    public double balance;

    public Account(String accountnumber, String accountholder, double balance) {
        this.accountnumber = accountnumber; // using constructors for parameters passing
        this.accountholder = accountholder;
        this.balance = balance;
    }

    public String getaccountnumber() { // doing encapsulation by setters and getters
        return accountnumber;
    }
}
```

```

public String getaccountholder() {
    return accountholder;
}

public double getbalance() {
    return balance;
}

void deposit(double amount) {
    if (amount <= 0) {
        System.out.println("deposit amount is greater than zero");
        return;
    }
    balance += amount;
    System.out.println(" the deposited amount is " + amount);
}

void withdraw(double amount) {}

void calinterest() {}

void transfer(Account fixedAccount, double amount) {
    System.out.println("transfer of" + amount + "from" + accountnumber + "to" +
fixedAccount.getaccountnumber() + "---");
    this.withdraw(amount);
    fixedAccount.deposit(amount);
}

void show() {
    System.out.println("ACCOUNT NUMBER IS " + accountnumber);
    System.out.println("ACCOUNT HOLDER IS " + accountholder);
    System.out.println("BALANCE IS " + balance);
}
}

class SavingsAccount extends Account {
    private double interestrate;

    public SavingsAccount(String accountnumber, String accountholder, double balance) {
        super(accountnumber, accountholder, balance);
        this.interestrate = interestrate;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("with draw amount is not possible ");
        } else if (amount >= balance) {
            System.out.println("it has insufficient balance");
        } else {
            balance -= amount;
            System.out.println("withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double interest = balance * interestrate;
        balance += interest;
        System.out.println("interest is credited" + interest);
    }
}

```

```

class CurrentAccount extends Account {
    private double overdraftlimit;

    public CurrentAccount(String accountnumber, String accountholder, double balance) {
        super(accountnumber, accountholder, balance);
        this.overdraftlimit = overdraftlimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0)
            System.out.println("with draw amount is not possible ");
        else if (amount > balance + overdraftlimit) {
            System.out.println("it has insufficient balance and the limited is done");
        } else {
            balance -= amount;
            System.out.println("withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double charge = 50.0;
        balance += charge;
        System.out.println("maintainence charge is credited" + charge);
    }
}

public class LabProgram1 {
    public static void main(String[] args) {
        CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500);
        SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000);

        System.out.println(" initial account details");
        ca.show();
        System.out.println("");
        sa.show();

        System.out.println("transaction details");
        ca.deposit(2000);
        ca.withdraw(2500);
        ca.calinterest();
        sa.withdraw(3000);
        sa.calinterest();
        sa.transfer(ca, 1000);

        System.out.println("final account details");
        ca.show();
        System.out.println("");
        sa.show();
    }
}

```

## 7. Program Output

Execution of the code yields the following output in the terminal:



## Terminal Output

```
initial account details
ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 4500.0

ACCOUNT NUMBER IS sb 24533
ACCOUNT HOLDER IS heamnth
BALANCE IS 5000.0
transaction details
the deposited amount is 2000.0
withdraw is 2500.0
maintainence charge is credited50.0
withdraw is 3000.0
interest is credited0.0
transfer of1000.0fromsb 24533toUB 24311---
withdraw is 1000.0
the deposited amount is 1000.0
final account details
ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 5050.0

ACCOUNT NUMBER IS sb 24533
ACCOUNT HOLDER IS heamnth
BALANCE IS 1000.0
```